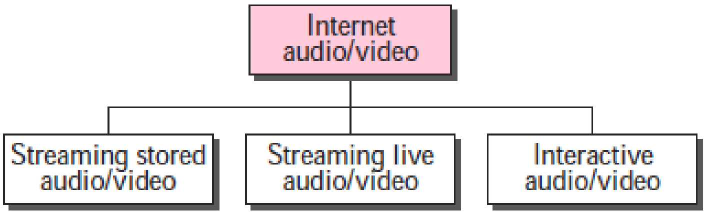
1. INTRODUCTION

We can divide audio and video services into three broad categories: **streaming stored audio/video, streaming live audio/video,** and **interactive audio/video**. Streaming means a user can listen (or watch) the file after the downloading has started.



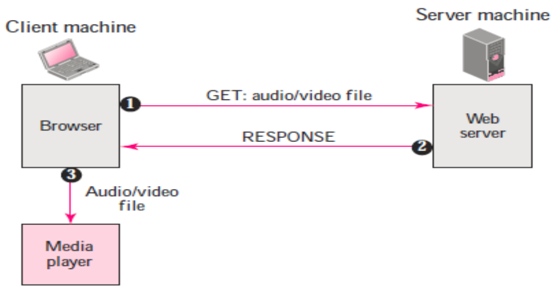
In the **first category**, streaming stored audio/video, **the files are compressed and stored on a server**. A client downloads the files through the Internet. This is sometimes referred to as **on-demand audio/video.** In the **second category**, streaming live audio/video **refers to the broadcasting of radio and TV programs through the Internet.** In the **third category**, interactive audio/video **refers to the use of the Internet for interactive audio/video applications.** A good example of this application is Internet telephony and Internet teleconferencing.

2. STREAMING STORED AUDIO/VIDEO

Downloading these types of files from a server can be different from downloading other types of files.

2.1. First Approach: Using a Web Server

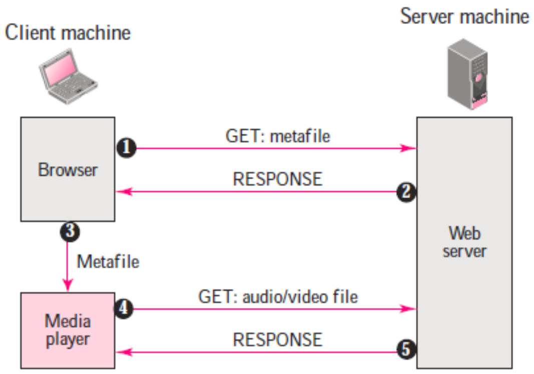
A compressed audio/video file can be downloaded as a text file. The **client (browser)** can use the services of **HTTP** and send a GET message to download the file. The **Web server** can send the compressed file to the browser. The browser can then use a help application, normally called a **media player**, to play the file. The file needs to download completely before it can be played.



2.2. Second Approach: Using a Web Server with Metafile

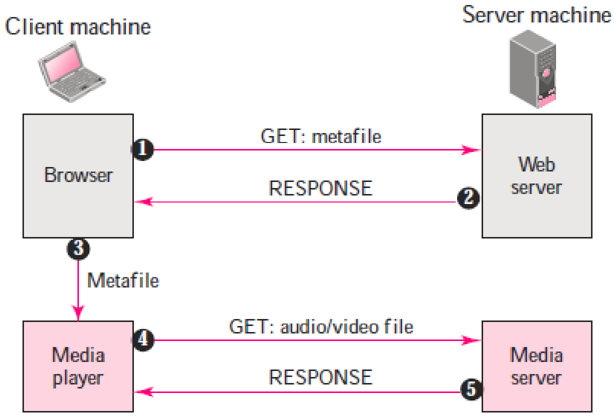
In another approach, the media player is directly connected to the Web server for downloading the audio/video file. The Web server stores two files: **the actual audio/video file** and a **metafile** that holds information about the audio/video file.

1. The HTTP client accesses the Web server using the GET message.
2. The information about the metafile comes in the response.
3. The metafile is passed to the media player.
4. The media player uses the URL in the metafile to access the audio/video file.
5. The Web server responds.



2.3. Third Approach: Using a Media Server

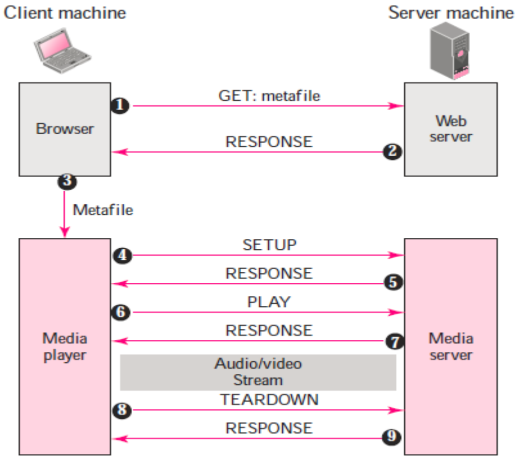
The **problem** with the **second approach** is that **the browser and the media player both use the services of HTTP**. HTTP is designed to run over TCP. This is appropriate for retrieving the metafile, but not for retrieving the audio/video file**. The reason is that TCP retransmits a lost or damaged segment, which is counter to the philosophy of streaming.** We need to dismiss TCP and its error control; we need to use UDP. However, HTTP, which accesses the Web server, and the Web server itself are designed for TCP; we need another server, a **media server.**



1. The HTTP client accesses the Web server using a GET message.
2. The information about the metafile comes in the response.
3. The metafile is passed to the media player.
4. The media player uses the URL in the metafile to access the media server to download the file. Downloading can take place by any protocol that uses UDP.
5. The media server responds.

2.4. Fourth Approach: Using a Media Server and RTSP

The **Real-Time Streaming Protocol (RTSP**) is a control protocol designed to add more functionalities to the streaming process. Using RTSP, we can control the playing of audio/video. Figure 5 shows a media server and RTSP.



1. The HTTP client accesses the Web server using a GET message.
2. The information about the metafile comes in the response.
3. The metafile is passed to the media player.
4. The media player sends a SETUP message to create a connection with the media server.
5. The media server responds.
6. The media player sends a PLAY message to start playing (downloading).
7. The audio/video file is downloaded using another protocol that runs over UDP.
8. The connection is broken using the TEARDOWN message.
9. The media server responds.

3. STREAMING LIVE AUDIO/VIDEO

Streaming live audio/video is similar to the broadcasting of audio and video by radio and TV stations. Instead of broadcasting to the air, the stations broadcast through the Internet. There are several similarities between streaming stored audio/video and streaming live audio/video. They are both sensitive to delay; neither can accept retransmission. However, there is a difference. In the first application, the communication is unicast and on-demand. In the second, the communication is multicast and live. Live streaming is better suited to the multicast services of IP and the use of protocols such as UDP and RTP.

Examples: Internet Radio, Internet Television (ITV), Internet protocol television (IPTV)

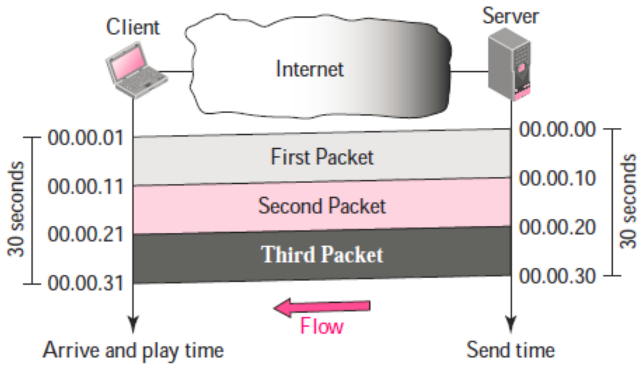
4. REAL-TIME INTERACTIVE AUDIO/VIDEO

In real-time interactive audio/video, people communicate with one another in real time. The Internet phone or voice over IP is an example of this type of application. Video conferencing is another example that allows people to communicate visually and orally.

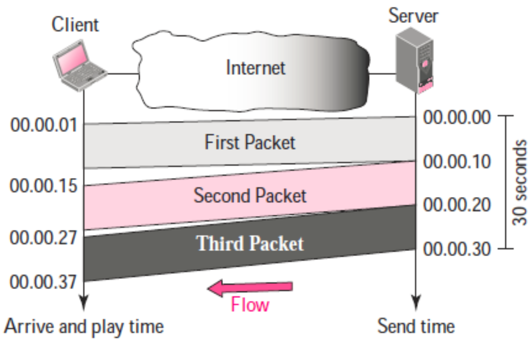
Before discussing the protocols used in this class of applications, we discuss some characteristics of real-time audio/video communication.

- **Time Relationship**

Real-time data on a packet-switched network **require the preservation of the time relationship between packets of a session.**

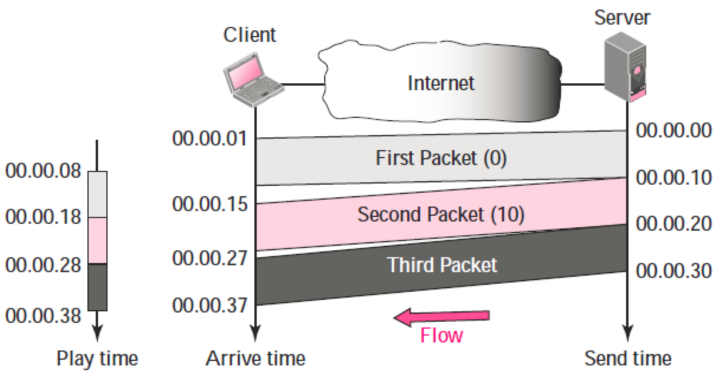


But what happens if the packets arrive with different delays? For example, the first packet arrives at 00:00:01 (1-s delay), the second arrives at 00:00:15 (5-s delay), and the third arrives at 00:00:27 (7-s delay). If the receiver starts playing the first packet at 00:00:01, it will finish at 00:00:11. However, the next packet has not yet arrived; it arrives 4 s later. There is a gap between the first and second packets and between the second and the third as the video is viewed at the remote site. This phenomenon is called **jitter**. **Jitter is introduced in real-time data by the delay between packets.**



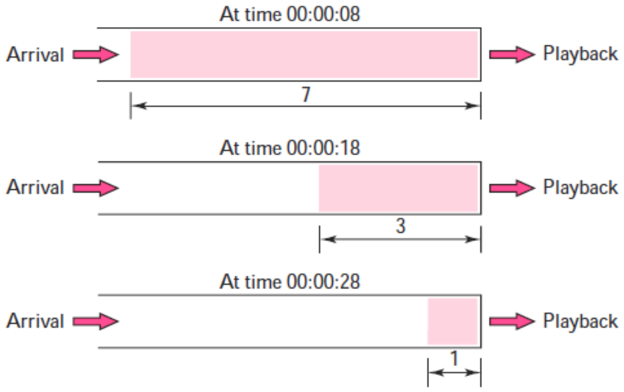
- **Timestamp**

One solution to **jitter** is the use of a **timestamp. If each packet has a timestamp that shows the time it was produced relative to the first (or previous) packet, then the receiver can add this time to the time at which it starts the playback.** Imagine the first packet in the previous example has a timestamp of 0, the second has a timestamp of 10, and the third a timestamp of 20. If the receiver starts playing back the first packet at 00:00:08, the second will be played at 00:00:18, and the third at 00:00:28. There are no gaps between the packets.



- **Playback Buffer**

To be able **to separate** the **arrival time** from the **playback time**, we need a buffer to store the data until they are played back. The buffer is referred to as a **playback buffer.** In the previous example, the first bit of the first packet arrives at 00:00:01; the threshold is 7 s, and the playback time is 00:00:08. The threshold is measured in time units of data. The replay does not start until the time units of data are equal to the threshold value.



- **Ordering**

We need a ***sequence number*** for each packet. The timestamp alone cannot inform the receiver if a packet is lost.

- **Multicasting**

Multimedia play a primary role in audio and video conferencing. The traffic can be heavy, and the data are distributed using **multicasting** methods. Conferencing requires two-way communication between receivers and senders.

- **Mixing**

If there is more than one source that can send data at the same time (as in a video or audio conference), the traffic is made of multiple streams. **Mixing means combining several streams of traffic into one stream.**

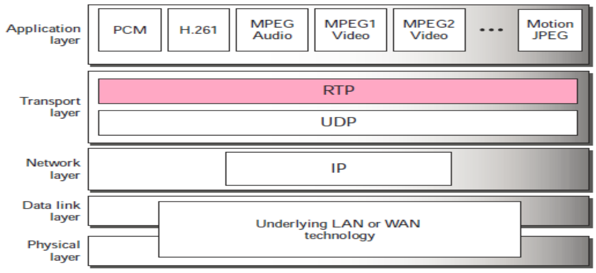
- ***Support from Transport Layer Protocol***

TCP is not suitable for interactive traffic. It has no provision for timestamping, and it does not support multicasting. However, it does provide ordering (sequence numbers). One feature of TCP that makes it particularly unsuitable for interactive traffic **is its error control mechanism**.

UDP is more suitable for interactive multimedia traffic. UDP supports multicasting and has no retransmission strategy. However, UDP has no provision for timestamping, sequencing, or mixing. A new transport protocol, **Real-Time Transport Protocol (RTP),** provides these missing features.

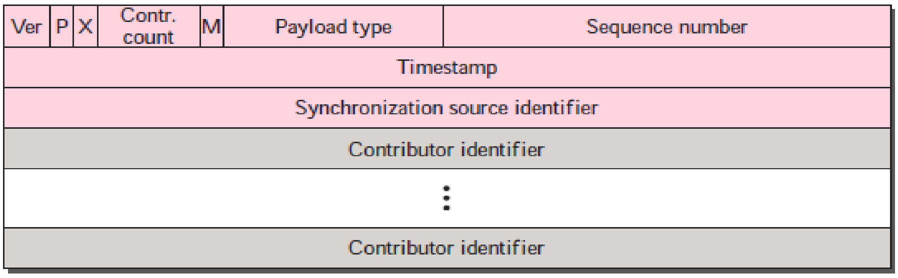
5. Real-time Transport Protocol (RTP)

Real-time Transport Protocol (RTP) is the protocol designed to handle real-time traffic on the Internet. RTP does not have a delivery mechanism (multicasting, port numbers, and so on); it must be used with UDP. RTP stands between UDP and the application program. The main contributions of RTP are timestamping, sequencing, and mixing facilities.



5.1. RTP Packet Format

The format is very simple and general enough to cover all real-time applications. An application that needs more information adds it to the beginning of its payload. A description of each field follows.



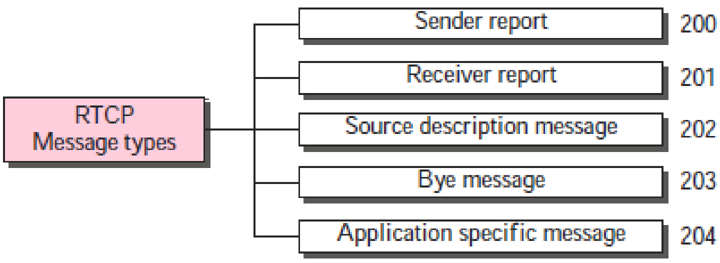
- **Ver.** This 2-bit field defines the version number.
- **P.** This 1-bit field, if set to 1, indicates the presence of padding at the end of the packet. There is no padding if the value of the P field is 0.
- **X.** This 1-bit field, if set to 1, indicates an extra extension header between the basic header and the data. There is no extra extension header if the value of this field is 0.
- **Contributor count.** This 4-bit field indicates the number of contributors. Note that we can have a maximum of 15 contributors because a 4-bit field only allows a number between 0 and 15.
- **M.** This 1-bit field is a marker used by the application to indicate, for example, the end of its data.
- **Payload type.** This 7-bit field indicates the type of the payload. Several payload types have been defined so far.
- **Sequence number.** This field is 16 bits in length. It is used to number the RTP packets. The sequence number of the first packet is chosen randomly; it is incremented by 1 for each subsequent packet. The sequence number is used by the receiver to detect lost or out of order packets.
- **Timestamp.** This is a 32-bit field that indicates the time relationship between packets.
- **Synchronization source identifier.** If there is only one source, this 32-bit field defines the source. However, if there are several sources, the mixer is the synchronization source and the other sources are contributors. The value of the source identifier is a random number chosen by the source. The protocol provides a strategy in case of conflict (two sources start with the same sequence number).
- **Contributor identifier.** Each of these 32-bit identifiers (a maximum of 15) defines a source. When there is more than one source in a session, the mixer is the synchronization source and the remaining sources are the contributors.

5.2. UDP Port

Although RTP is itself a transport layer protocol, the RTP packet is not encapsulated directly in an IP datagram. Instead, RTP is treated like an application program and is encapsulated in a UDP user datagram. However, unlike other application programs, **no well-known port is assigned to RTP**. The port can be selected on demand with only one restriction: The port number must be an even number. The next number (an odd number) is used by the companion of RTP, Real-Time Transport Control Protocol (RTCP).

6. Real-Time Transport Control Protocol (RTCP)

RTP allows only one type of message, one that carries data from the source to the destination. In many cases, there is a need for other messages in a session. **These messages control the flow and quality of data and allow the recipient to send feedback to the source or sources**. **Real-Time Transport Control Protocol (RTCP)** is a protocol designed for this purpose. RTCP has five types of messages, as shown in the following Figure. The number next to each box defines the type of the message.



6.1. Types of messages

- Sender Report:

  The sender report is sent periodically by the active senders in a conference to report transmission and reception statistics for all RTP packets sent during the interval.
- Receiver Report:

  The receiver report is for passive participants, those that do not send RTP packets. The report informs the sender and other receivers about the quality of service.
- Source Description Message:

  The source periodically sends a source description message to give additional information about itself.
- Bye Message:

  A source sends a bye message to shut down a stream. It allows the source to announce that it is leaving the conference.
- Application-Specific Message

  The application-specific message is a packet for an application that wants to use new applications (not defined in the standard). It allows the definition of a new message type.

6.2. UDP Port

RTCP, like RTP, does not use a well-known UDP port. It uses a temporary port. The UDP port chosen must be the number immediately following the UDP port selected for RTP, which makes it an odd-numbered port.

7. VOICE OVER IP (Real-time interactive audio/video application)

The idea is to use the Internet as a telephone network with some additional capabilities. Instead of communicating over a circuit-switched network, this application allows communication between two parties over the packet-switched Internet. Two protocols have been designed to handle this type of communication: SIP(Session Initiation Protocol) and H.323.